

Task 61: Course Schedule II

Problem Description

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`. Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Java Solution

```
class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        int[] indegree = new int[numCourses];
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) {
            adj.add(new ArrayList<>());
        }

        for (int[] pre : prerequisites) {
            adj.get(pre[1]).add(pre[0]);
            indegree[pre[0]]++;
        }

        Queue<Integer> q = new LinkedList<>();
        for (int i = 0; i < numCourses; i++) {
            if (indegree[i] == 0) q.offer(i);
        }

        int[] order = new int[numCourses];
        int index = 0;
        while (!q.isEmpty()) {
            int curr = q.poll();
            order[index++] = curr;
            for (int next : adj.get(curr)) {
                indegree[next]--;
                if (indegree[next] == 0) {
                    q.offer(next);
                }
            }
        }

        return index == numCourses ? order : new int[0];
    }
}
```